

Operaciones con Cadenas de Texto

1. `lower()`: Convierte todos los caracteres de una cadena a minúsculas.

Ejemplo:

```
texto = 'SmArTpHoNe'  
  
print(texto.lower()) # smartphone
```

2. `upper()`: Convierte todos los caracteres de una cadena a mayúsculas.

Ejemplo:

```
texto = 'SmArTpHoNe'  
  
print(texto.upper()) # SMARTPHONE
```

3. `capitalize()`: Convierte la primera letra de la cadena a mayúscula y el resto a minúsculas.

Ejemplo:

```
texto = "happy birthday"  
  
print(texto.capitalize()) # Happy birthday
```

4. `find()`: Busca la primera aparición de un carácter o subcadena. Devuelve el índice de la primera aparición o -1 si no se encuentra.

Ejemplo:

```
texto = 'roBot'  
  
print(texto.find("o")) # 1  
  
print(texto.find("z")) # -1
```

Operaciones con Listas

1. Longitud de la lista (len()): Devuelve el número de elementos en una lista.

Ejemplo:

```
peliculas = ["Avatar", "Titanic", "Avengers"]  
  
print(len(peliculas)) # 3
```

2. append(): Añade un elemento al final de la lista.

Ejemplo:

```
canciones = ["Yesterday", "Hello", "Believer"]  
  
canciones.append("Imagine")  
  
print(canciones) # ['Yesterday', 'Hello', 'Believer', 'Imagine']
```

3. insert(): Inserta un elemento en una posición específica de la lista.

Ejemplo:

```
elementos = ["book", "pen", "pencil"]  
  
elementos.insert(2, "marker") # Inserta "marker" en la posición 2  
  
print(elementos) # ['book', 'pen', 'marker', 'pencil']
```

4. pop(): Elimina y devuelve el elemento de una posición específica en la lista.

Ejemplo:

```
elementos = ["book", "pen", "pencil"]  
  
print(elementos.pop(1)) # Elimina y devuelve "pen"  
  
print(elementos) # ['book', 'pencil']
```

5. Ordenar una lista (sorted()): Devuelve una nueva lista con los elementos ordenados en orden ascendente (por defecto).

Ejemplo:

```
edades = [25, 36, 33, 19, 56]
```

```
print(sorted(edades)) # [19, 25, 33, 36, 56]
```

6. `sum()`, `max()`, `min()`:

- `sum()`: Suma todos los elementos de la lista.

- `max()`: Devuelve el valor máximo.

- `min()`: Devuelve el valor mínimo.

Ejemplo:

```
edades = [25, 36, 33, 19, 56]
```

```
print(sum(edades)) # 169
```

```
print(max(edades)) # 56
```

```
print(min(edades)) # 19
```

Operaciones con Tuplas

1. `count()`: Cuenta cuántas veces aparece un valor en una tupla.

Ejemplo:

```
puntuaciones = (7, 9, 9, 8, 9)
```

```
print(puntuaciones.count(7)) # 1
```

```
print(puntuaciones.count(9)) # 3
```

2. `index()`: Devuelve el índice de la primera aparición de un valor. Si no se encuentra, lanza un error.

Ejemplo:

```
print(puntuaciones.index(9)) # 1
```

Operaciones con Diccionarios

1. Estructura de un Diccionario: Los diccionarios almacenan datos en pares de clave-valor.

Ejemplo:

```
contacto = {  
    "nombre": "Juan",  
    "numero": 6496426  
}
```

2. Acceder a valores con claves: Puedes acceder a los valores de un diccionario mediante la clave o usando el método `get()`.

Ejemplo:

```
print(contacto["nombre"]) # Juan  
print(contacto.get("numero")) # 6496426
```

3. Modificar o agregar elementos: Puedes actualizar valores o añadir nuevos elementos al diccionario.

Ejemplo:

```
contacto["direccion"] = "Calle 123"  
print(contacto) # {'nombre': 'Juan', 'numero': 6496426, 'direccion': 'Calle 123'}
```

4. Eliminar elementos: Puedes eliminar elementos con `pop()` o `del`.

Ejemplo:

```
contacto.pop("direccion")  
print(contacto) # {'nombre': 'Juan', 'numero': 6496426}
```

5. Recuperar claves, valores e ítems:

- `keys()`: Devuelve las claves del diccionario.
- `values()`: Devuelve los valores del diccionario.
- `items()`: Devuelve los pares clave:valor.

Ejemplo:

```
print(contacto.keys()) # dict_keys(['nombre', 'numero'])  
print(contacto.values()) # dict_values(['Juan', 6496426])  
print(contacto.items()) # dict_items([('nombre', 'Juan'), ('numero', 6496426)])
```

Funciones y Expresiones Lambda

1. Funciones Básicas: Las funciones en Python son definidas con la palabra clave `def`. Pueden tomar parámetros y devolver resultados.

Ejemplo:

```
def bienvenida(nombre):  
    return "Bienvenido, " + nombre  
print(bienvenida("Juan")) # Bienvenido, Juan
```

2. Funciones de Orden Superior: Puedes pasar funciones como argumentos.

Ejemplo:

```
def procesar_usuario(nombre, funcion):  
    return funcion(nombre)  
print(procesar_usuario("Alice", bienvenida)) # Bienvenido, Alice
```

3. Expresiones Lambda: Las funciones anónimas se definen usando `lambda` y pueden ser útiles para tareas rápidas.

Ejemplo:

```
saludo = lambda nombre: "Hola, " + nombre
```

```
print(saludo("Bob")) # Hola, Bob
```

4. Lambda con funciones dentro de otras funciones: Puedes usar lambdas para crear funciones dinámicamente.

Ejemplo:

```
def multiplicar(n):
```

```
    return lambda a: a * n
```

```
duplicar = multiplicar(2)
```

```
triplicar = multiplicar(3)
```

```
print(duplicar(5)) # 10
```

```
print(triplicar(5)) # 15
```

***args y **kwargs**

1. *args: Permite pasar un número variable de argumentos posicionales. Estos se almacenan en una tupla.

Ejemplo:

```
def suma(*args):
```

```
    return sum(args)
```

```
print(suma(1, 2, 3, 4)) # 10
```

2. **kwargs: Permite pasar un número variable de argumentos con nombre (clave: valor). Estos se almacenan en un diccionario.

Ejemplo:

```
def mostrar_info(**kwargs):
```

```
    for clave, valor in kwargs.items():
```

```
print(f'{clave}: {valor}')
```

```
mostrar_info(nombre="Juan", edad=25)
```

```
# nombre: Juan
```

```
# edad: 25
```